

Informe Laboratorio 5

Sección 2

Alumno Camilo Ríos
e-mail: camilo.rios1@mail.udp.cl

Noviembre de 2025

Índice

Descripción de actividades	3
1. Desarrollo (Parte 1)	6
1.1. Códigos de cada Dockerfile	6
1.1.1. C1	6
1.1.2. C2	6
1.1.3. C3	6
1.1.4. C4/S1	6
1.2. Creación de las credenciales para S1	7
1.3. Tráfico generado por C1, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	7
1.4. Tráfico generado por C2, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	8
1.5. Tráfico generado por C3, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	10
1.6. Tráfico generado por C4 (iface lo), detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	11
1.7. Tipo de información contenida en cada uno de los paquetes generados en texto plano	13
1.7.1. C1	13
1.7.2. C2	13
1.7.3. C3	13
1.7.4. C4/S1	13
1.8. Diferencia entre C1 y C2	13
1.9. Diferencia entre C2 y C3	14
1.10. Diferencia entre C3 y C4	14
2. Desarrollo (Parte 2)	15
2.1. Identificación del cliente SSH con versión “?”	15
2.2. Replicación de tráfico al servidor (paso por paso)	15
3. Desarrollo (Parte 3)	16
3.1. Replicación del KEI con tamaño menor a 300 bytes (paso por paso)	16
4. Desarrollo (Parte 4)	19
4.1. Explicación OpenSSH en general	19
4.2. Capas de Seguridad en OpenSSH	19
4.3. Identificación de que protocolos no se cumplen	20

Descripción de actividades

Para este último laboratorio, se solicita trabajar con Docker y el protocolo SSH, a fin de poder entender el concepto de criptografía asimétrica y firmas digitales.

Para lo anterior deberá:

- Crear 4 contenedores en Docker por medio de un DockerFile, donde cada uno tendrá el siguiente SO: Ubuntu 16.10, Ubuntu 18.10, Ubuntu 20.10 y Ubuntu 22.10 a los cuales se llamarán C1, C2, C3 y C4 respectivamente.
El equipo con Ubuntu 22.10 también será utilizado como S1.
- Para cada uno de ellos, deberá instalar el cliente openSSH disponible en los repositorios de apt, y para el equipo S1 deberá también instalar el servidor openSSH.
- En S1 deberá crear el usuario “**prueba**” con contraseña “**prueba**”, para acceder a él desde los clientes por el protocolo SSH.
- En total serán 4 escenarios, donde cada uno corresponderá a los siguientes equipos:
 - C1 → S1
 - C2 → S1
 - C3 → S1
 - C4 → S1

Pasos:

1. Para cada uno de los 4 escenarios, solo deberá establecer la conexión y no realizar ningún otro comando que pueda generar tráfico (como muestra la Figura). Deberá capturar el tráfico de red generado y analizar el patrón de tráfico generado por cada cliente. De esta forma podrá obtener una huella digital para cada cliente a partir de su tráfico.

Indique el tamaño de los paquetes del flujo generados por el cliente y el contenido asociado a cada uno de ellos. Indique qué información distinta contiene el escenario siguiente (diff incremental). El objetivo de este paso es identificar claramente los cambios entre las distintas versiones de ssh.

2. Para poder identificar que el usuario efectivamente es el informante, éste utilizará una versión única de cliente. ¿Con qué cliente SSH se habrá generado el siguiente tráfico?

Protocol	Length	Info
TCP	74	34328 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=14
TCP	66	34328 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0
SSHv2	85	Client: Protocol (SSH-2.0-OpenSSH_?)
TCP	66	34328 → 22 [ACK] Seq=20 Ack=42 Win=64256 Len=
SSHv2	1578	Client: Key Exchange Init
TCP	66	34328 → 22 [ACK] Seq=1532 Ack=1122 Win=64128
SSHv2	114	Client: Elliptic Curve Diffie-Hellman Key Exc
TCP	66	34328 → 22 [ACK] Seq=1580 Ack=1574 Win=64128
SSHv2	82	Client: New Keys
SSHv2	110	Client: Encrypted packet (len=44)
TCP	66	34328 → 22 [ACK] Seq=1640 Ack=1618 Win=64128
SSHv2	126	Client: Encrypted packet (len=60)
TCP	66	34328 → 22 [ACK] Seq=1700 Ack=1670 Win=64128
SSHv2	150	Client: Encrypted packet (len=84)
TCP	66	34328 → 22 [ACK] Seq=1784 Ack=1698 Win=64128
SSHv2	178	Client: Encrypted packet (len=112)
TCP	66	34328 → 22 [ACK] Seq=1896 Ack=2198 Win=64128

Figura 1: Tráfico generado del informante

Replique este tráfico generado en la imagen. Debe generar el tráfico con la misma versión resaltada en azul. Recuerde que toda la información generada es parte del sw, por lo tanto usted puede modificar toda la información.

3. Para que el informante esté seguro de nuestra identidad, nos pide que el patrón del tráfico de nuestro server también sea modificado, hasta que el Key Exchange Init del server sea menor a 300 bytes. Indique qué pasos realizó para lograr esto.

TCP	66	42350 → 22	[ACK]	Seq=2	Ack=
TCP	74	42398 → 22	[SYN]	Seq=0	Win=
TCP	74	22 → 42398	[SYN, ACK]	Seq=0	
TCP	66	42398 → 22	[ACK]	Seq=1	Ack=
SSHv2	87	Client: Protocol (SSH-2.0-C			
TCP	66	22 → 42398	[ACK]	Seq=1	Ack=
SSHv2	107	Server: Protocol (SSH-2.0-C			
TCP	66	42398 → 22	[ACK]	Seq=22	Ack=
SSHv2	1570	Client: Key Exchange Init			
TCP	66	22 → 42398	[ACK]	Seq=42	Ack=
SSHv2	298	Server: Key Exchange Init			
TCP	66	42398 → 22	[ACK]	Seq=1526	A

Figura 2: Captura del Key Exchange

- Tomando en cuenta lo aprendido en este laboratorio, así como en los anteriores, explique el protocolo OpenSSH y las diferentes capas de seguridad que son parte del protocolo para garantizar los principios de seguridad de la información, integridad, confidencialidad, disponibilidad, autenticidad y no repudio. Es importante que sea muy específico en el objetivo del principio en el protocolo. En caso de considerar que alguno de los principios no se cumple, justifique su razonamiento. Es fundamental que su análisis se base en el tráfico SSH interceptado.

1. Desarrollo (Parte 1)

1.1. Códigos de cada Dockerfile

Se crean los Dockerfile de los clientes C1, C2 y C3. Además se crea el Dockerfile de C4-S1 que actuará como cliente y servidor a la vez. Para las imágenes de los contenedores se utiliza la página Server Pilot que indica las versiones de Ubuntu compatibles con el protocolo OpenSSH, en este caso no se utilizan las versiones indicadas en el laboratorio ya que son incompatibles con el cliente y servidor OpenSSH, finalmente se reemplazan las versiones menores XX.10 por las versiones XX.04 manteniendo la versión mayor solicitada.

1.1.1. C1

```
1 FROM ubuntu:16.04
2 RUN apt-get update && apt-get install -y openssh-client tcpdump
3 CMD ["tail", "-f", "/dev/null"]
```

Listing 1: Dockerfile - C1 (Ubuntu 16.04)

1.1.2. C2

```
1 FROM ubuntu:18.04
2 RUN apt-get update && apt-get install -y openssh-client tcpdump
3 CMD ["tail", "-f", "/dev/null"]
```

Listing 2: Dockerfile - C2 (Ubuntu 18.04)

1.1.3. C3

```
1 FROM ubuntu:20.04
2 RUN apt-get update && apt-get install -y openssh-client tcpdump
3 CMD ["tail", "-f", "/dev/null"]
```

Listing 3: Dockerfile - C3 (Ubuntu 20.04)

1.1.4. C4/S1

```
1 FROM ubuntu:22.04
2 RUN apt-get update && apt-get install -y openssh-client openssh-server tcpdump
3 RUN mkdir -p /var/run/sshd && \
4     sed -i 's/#PasswordAuthentication yes/PasswordAuthentication yes/' /etc/ssh/
5     sshd_config
6 RUN ssh-keygen -A
7 EXPOSE 22
```

```
7 CMD ["/usr/sbin/sshd", "-D"]
```

Listing 4: Dockerfile - C4-S1 (Ubuntu 22.04)

1.2. Creación de las credenciales para S1

Para crear las credenciales del usuario prueba con contraseña prueba en el contenedor C4-S1 se ejecuta el comando `docker exec -it c4-s1 bash` para acceder a la terminal del contenedor y luego se ejecuta el comando `useradd -m -s /bin/bash prueba && echo 'prueba:prueba' | chpasswd` para crear el usuario con la contraseña indicada. Luego, para verificar que el usuario se crea correctamente se utiliza el comando `su - prueba` y se ingresa la contraseña *prueba*, obteniendo el siguiente resultado:

```
camilo@DESKTOP-MJ6DSQJ:~/udp/cryptografia/lab5$ sudo docker exec -it C4-S1 bash
root@bfa419527443:/# useradd -m -s /bin/bash prueba
root@bfa419527443:/# echo 'prueba:prueba' | chpasswd
root@bfa419527443:/# su - prueba
prueba@bfa419527443:~$
```

Figura 3: Creación del usuario prueba en C4-S1.

1.3. Tráfico generado por C1, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Se captura el tráfico ssh de C1 al conectarse a C4-S1 y se obtiene lo siguiente:

8 0.000516	172.20.0.5	172.20.0.3	TCP	76 36252 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK PERM TSval=3806109711 TSecr=0 WS=128
9 0.000560	172.20.0.3	172.20.0.5	TCP	76 22 → 36252 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK PERM TSval=3912540029 TSecr=
10 0.000572	172.20.0.5	172.20.0.3	TCP	68 36252 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3806109711 TSecr=3912540029
11 0.002484	172.20.0.5	172.20.0.3	SSHv2	110 Client: Protocol (SSH-2.0-OpenSSH 7.2p2 Ubuntu-4ubuntu2.10)
12 0.002508	172.20.0.3	172.20.0.5	TCP	68 22 → 36252 [ACK] Seq=1 Ack=43 Win=65152 Len=0 TSval=3912540031 TSecr=3806109713
13 0.018274	172.20.0.3	172.20.0.5	SSHv2	110 Server: Protocol (SSH-2.0-OpenSSH 8.9p1 Ubuntu-3ubuntu0.13)
14 0.018293	172.20.0.5	172.20.0.3	TCP	68 36252 → 22 [ACK] Seq=43 Ack=43 Win=64256 Len=0 TSval=3806109729 TSecr=3912540046
15 0.018839	172.20.0.5	172.20.0.3	SSHv2	1404 Client: Key Exchange Init
16 0.020719	172.20.0.3	172.20.0.5	SSHv2	1180 Server: Key Exchange Init
17 0.021998	172.20.0.5	172.20.0.3	SSHv2	116 Client: Elliptic Curve Diffie-Hellman Key Exchange Init
18 0.024941	172.20.0.3	172.20.0.5	SSHv2	664 Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys
19 0.069302	172.20.0.5	172.20.0.3	TCP	68 36252 → 22 [ACK] Seq=1427 Ack=1751 Win=70144 Len=0 TSval=3806109780 TSecr=3912540053
20 1.982530	172.20.0.5	172.20.0.3	SSHv2	84 Client: New Keys
21 2.025348	172.20.0.3	172.20.0.5	TCP	68 22 → 36252 [ACK] Seq=1751 Ack=1443 Win=68096 Len=0 TSval=3912542054 TSecr=3806111693
22 2.025366	172.20.0.5	172.20.0.3	SSHv2	112 Client:
23 2.025407	172.20.0.3	172.20.0.5	TCP	68 22 → 36252 [ACK] Seq=1751 Ack=1487 Win=68096 Len=0 TSval=3912542054 TSecr=3806111736
24 2.025479	172.20.0.3	172.20.0.5	SSHv2	112 Server:
25 2.025504	172.20.0.5	172.20.0.3	TCP	68 36252 → 22 [ACK] Seq=1487 Ack=1795 Win=70144 Len=0 TSval=3806111736 TSecr=3912542054
26 2.025551	172.20.0.5	172.20.0.3	SSHv2	136 Client:
27 2.034213	172.20.0.3	172.20.0.5	SSHv2	120 Server:
28 2.077315	172.20.0.5	172.20.0.3	TCP	68 36252 → 22 [ACK] Seq=1555 Ack=1847 Win=70144 Len=0 TSval=3806111788 TSecr=3912542062
29 4.637198	172.20.0.5	172.20.0.3	SSHv2	216 Client:
30 4.681196	172.20.0.3	172.20.0.5	TCP	68 22 → 36252 [ACK] Seq=1847 Ack=1703 Win=71040 Len=0 TSval=3912544710 TSecr=3806114348
31 4.706316	172.20.0.3	172.20.0.5	SSHv2	96 Server:
32 4.706347	172.20.0.5	172.20.0.3	TCP	68 36252 → 22 [ACK] Seq=1703 Ack=1875 Win=70144 Len=0 TSval=3806114417 TSecr=3912544735
33 4.706450	172.20.0.5	172.20.0.3	SSHv2	180 Client:
34 4.706475	172.20.0.3	172.20.0.5	TCP	68 22 → 36252 [ACK] Seq=1875 Ack=1815 Win=71040 Len=0 TSval=3912544735 TSecr=3806114417
35 4.734903	172.20.0.3	172.20.0.5	SSHv2	696 Server:
36 4.777172	172.20.0.5	172.20.0.3	TCP	68 36252 → 22 [ACK] Seq=1815 Ack=2503 Win=71424 Len=0 TSval=3806114488 TSecr=3912544763
37 4.777217	172.20.0.3	172.20.0.5	SSHv2	112 Server:
38 4.777241	172.20.0.5	172.20.0.3	TCP	68 36252 → 22 [ACK] Seq=1815 Ack=2547 Win=71424 Len=0 TSval=3806114488 TSecr=3912544806
39 4.777316	172.20.0.5	172.20.0.3	SSHv2	444 Client:
40 4.784257	172.20.0.3	172.20.0.5	SSHv2	176 Server:
41 4.784480	172.20.0.3	172.20.0.5	SSHv2	800 Server:
42 4.784516	172.20.0.5	172.20.0.3	TCP	68 36252 → 22 [ACK] Seq=2191 Ack=3387 Win=71424 Len=0 TSval=3806114495 TSecr=3912544813
43 4.800485	172.20.0.3	172.20.0.5	SSHv2	160 Server:
44 4.841160	172.20.0.5	172.20.0.3	TCP	68 36252 → 22 [ACK] Seq=2191 Ack=3479 Win=71424 Len=0 TSval=3806114552 TSecr=3912544829

Frame 11: 110 bytes on wire (880 bits), 110 bytes captured (880 bit)	0000 00 04 00 01 00 06 56 ca 22 c1 55 4d 00 00 08 00	V. ".UM....
Linux cooked capture v1	0010 45 00 00 5e 41 ee 40 00 40 06 a0 7b ac 14 00 05	E...^A.@: @...{....
Internet Protocol Version 4, Src: 172.20.0.5, Dst: 172.20.0.3	0020 ac 14 00 03 8d 9c 00 16 7f 78 6e e9 35 5e 00 be	xn.5^...
Transmission Control Protocol, Src Port: 36252, Dst Port: 22, Seq:	0030 80 18 01 f6 58 81 00 00 01 01 08 0a e2 dc a0 11	X.....
SSH Protocol	0040 e9 34 9f 7d 53 53 48 2d 32 2e 30 2d 4f 70 65 6e	4JSSH- 2.0-Open
	0050 53 53 48 31 37 2e 32 70 32 20 55 62 75 6a 74 75	SSH 7.2p 2 Ubuntu
	0060 2d 34 75 62 75 6e 74 75 32 2e 31 38 0d 0a	4ubuntu 2.18..

Figura 4: Tráfico obtenido al realizar conexión ssh.

En la captura se puede ver como se realiza la conexión. Primero el cliente indica que tiene la versión de OpenSSH 7.2p2 y el servidor responde que tiene la versión de OpenSSH 8.9p1, luego ambos intercambian la lista de algoritmos de cifrado que tienen disponibles y eligen el primero que tengan en común para realizar el proceso de Diffie-Hellman con sus llaves y parámetros públicos. El HASSH es utilizado por servidor para identificar clientes SSH como Putty, OpenSSH, etc. y se calcula en base a la lista de algoritmos que entrega el cliente, es útil para identificar clientes desconocidos y por tanto poco confiables. Si se inspecciona el paquete Key Exchange Init que contiene el HASSH del cliente se obtiene lo siguiente:

```

15 0.018839 172.20.0.5 172.20.0.3 SSHv2 1404 Client: Key Exchange Init
16 0.020719 172.20.0.3 172.20.0.5 SSHv2 1180 Server: Key Exchange Init
Frame 15: 1404 bytes on wire (11232 bits), 1404 bytes captured (11232 bits)
Linux cooked capture v1
Internet Protocol Version 4, Src: 172.20.0.5, Dst: 172.20.0.3
Transmission Control Protocol, Src Port: 36252, Dst Port: 22, Seq: 43, Ack: 43,
SSH Protocol
  SSH Version 2 (encryption:chacha20-poly1305@openssh.com compression:none)
    Packet Length: 1332
    Padding Length: 5
    Key Exchange (method:curve25519-sha256@libssh.org)
      Message Code: Key Exchange Init (20)
      Algorithms
        Cookie: a5f026f750eb7f8edc6ec88b60f32628
        kex_algorithms length: 196
        kex_algorithms string: curve25519-sha256@libssh.org,ecdh-sha2-nistp2
        server_host_key_algorithms length: 290
        server_host_key_algorithms string [truncated]: ecdsa-sha2-nistp256-c
        encryption_algorithms_client_to_server length: 150
        encryption_algorithms_client_to_server string: chacha20-poly1305@ope
        encryption_algorithms_server_to_client length: 150
        encryption_algorithms_server_to_client string: chacha20-poly1305@ope
        mac_algorithms_client_to_server length: 213
        mac_algorithms_client_to_server string [truncated]: umac-64-etm@open
        mac_algorithms_server_to_client length: 213
        mac_algorithms_server_to_client string [truncated]: umac-64-etm@open
        compression_algorithms_client_to_server length: 26
        compression_algorithms_client_to_server string: none,zlib@openssh.co
        compression_algorithms_server_to_client length: 26
        compression_algorithms_server_to_client string: none,zlib@openssh.co
        languages_client_to_server length: 0
        languages_client_to_server string:
        languages_server_to_client length: 0
        languages_server_to_client string:
        First KEX Packet Follows: 0
        Reserved: 00000000
        [hasshAlgorithms [truncated]: curve25519-sha256@libssh.org,ecdh-sha2
        hassh: 68e0ba85e1a818f7c49ea3f4b849bd15

```

Figura 5: HASSH C1.

Como se observa, el paquete tiene un total de 1408 Bytes, mientras que la respuesta del servidor tiene un total de 1180 Bytes. Este HASSH debería ser idéntico para todas las KEI que se realicen con la lista de algoritmos entregada por la versión de OpenSSH.

1.4. Tráfico generado por C2, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Se captura el tráfico ssh de C2 al conectarse a C4-S1 y se obtiene lo siguiente:

1.4 Tráfico generado por C2, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

1 DESARROLLO (PARTE 1)

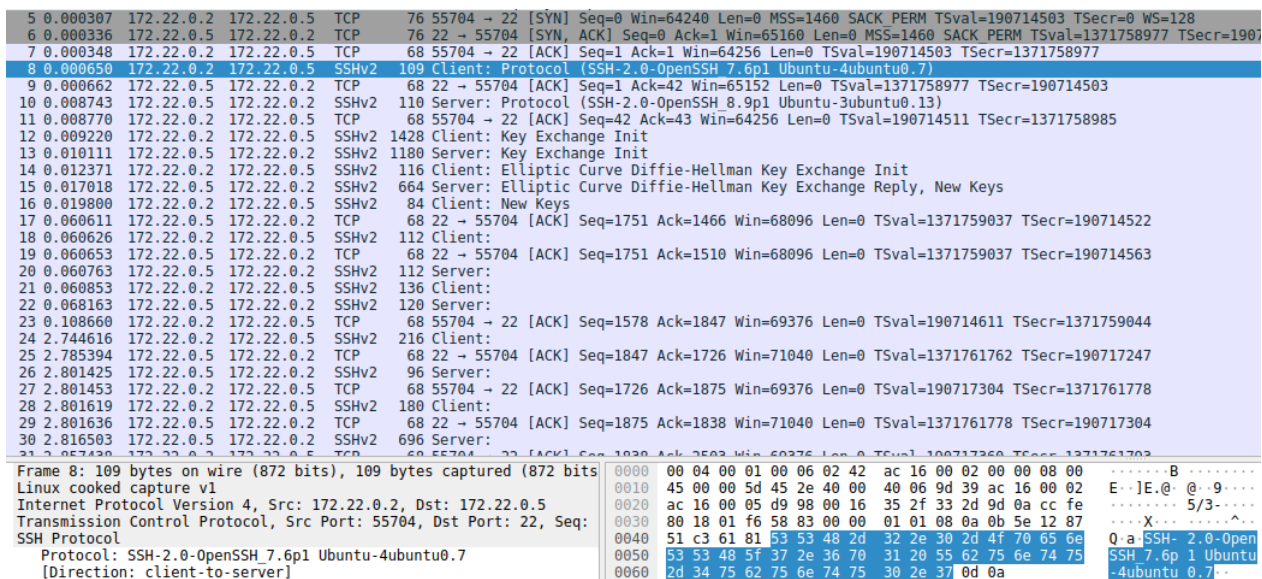


Figura 6: Tráfico obtenido al realizar conexión ssh.

En la captura se observa que se mantiene el funcionamiento explicado anteriormente, cambiando la versión OpenSSH del cliente, no así la del servidor ya que es el mismo. Si se inspecciona el paquete Key Exchange Init que contiene el HASSH del cliente se obtiene lo siguiente:

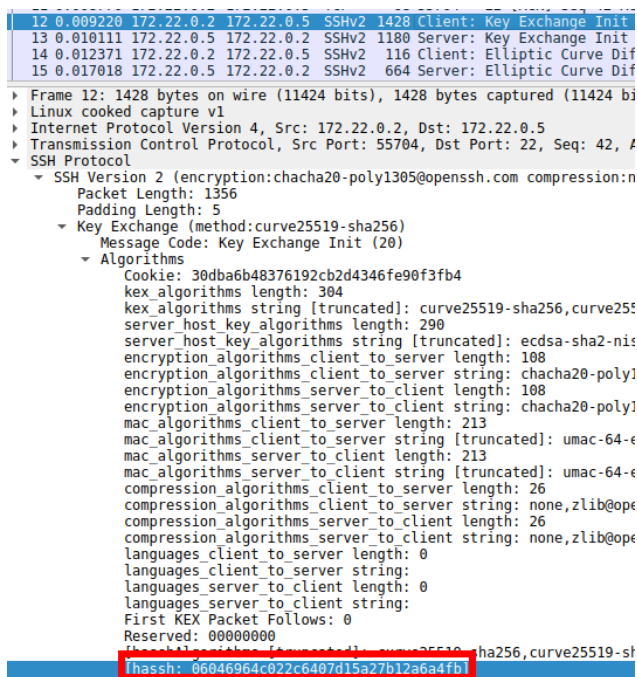


Figura 7: HASSH C2.

1.5 Tráfico generado por C3, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

1 DESARROLLO (PARTE 1)

Como se observa, el paquete tiene un total de 1428 Bytes, mientras que la respuesta del servidor tiene un total de 1180 Bytes. Este HASSH debería ser idéntico para todas las KEI que se realicen con la lista de algoritmos entregada por la versión de OpenSSH.

1.5. Tráfico generado por C3, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Se captura el tráfico ssh de C3 al conectarse a C4-S1 y se obtiene lo siguiente:

7	0.000451	172.22.0.4	172.22.0.5	TCP	76	44936 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK PERM TSval=4233800817 TSecr=0 WS=128
8	0.000474	172.22.0.5	172.22.0.4	TCP	76	22 → 44936 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK PERM TSval=1398588351 TSecr=4233
9	0.000490	172.22.0.4	172.22.0.5	TCP	68	44936 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=4233800817 TSecr=1398588351
10	0.001535	172.22.0.4	172.22.0.5	SSHv2	110	Client: Protocol (SSH-2.0-OpenSSH 8.2p1 Ubuntu-4ubuntu0.13)
11	0.001551	172.22.0.5	172.22.0.4	TCP	68	22 → 44936 [ACK] Seq=1 Ack=43 Win=65152 Len=0 TSval=1398588352 TSecr=4233800818
12	0.010549	172.22.0.5	172.22.0.4	SSHv2	110	Server: Protocol (SSH-2.0-OpenSSH 8.9p1 Ubuntu-3ubuntu0.13)
13	0.010584	172.22.0.4	172.22.0.5	TCP	68	44936 → 22 [ACK] Seq=43 Ack=43 Win=64256 Len=0 TSval=4233800827 TSecr=1398588361
14	0.011049	172.22.0.4	172.22.0.5	SSHv2	1604	Client: Key Exchange Init
15	0.011063	172.22.0.5	172.22.0.4	TCP	68	22 → 44936 [ACK] Seq=43 Ack=1579 Win=68224 Len=0 TSval=1398588361 TSecr=4233800827
16	0.011850	172.22.0.5	172.22.0.4	SSHv2	1180	Server: Key Exchange Init
17	0.013858	172.22.0.4	172.22.0.5	SSHv2	116	Client: Elliptic Curve Diffie-Hellman Key Exchange Init
18	0.016823	172.22.0.5	172.22.0.4	SSHv2	664	Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys
19	0.057281	172.22.0.4	172.22.0.5	TCP	68	44936 → 22 [ACK] Seq=1627 Ack=1751 Win=69376 Len=0 TSval=4233800874 TSecr=1398588367
20	2.719529	172.22.0.4	172.22.0.5	SSHv2	84	Client: New Keys
21	2.760403	172.22.0.5	172.22.0.4	TCP	68	22 → 44936 [ACK] Seq=1751 Ack=1643 Win=68224 Len=0 TSval=1398591111 TSecr=4233803536
22	2.760422	172.22.0.4	172.22.0.5	SSHv2	112	Client:
23	2.760455	172.22.0.5	172.22.0.4	TCP	68	22 → 44936 [ACK] Seq=1751 Ack=1687 Win=68224 Len=0 TSval=1398591111 TSecr=4233803577
24	2.760576	172.22.0.5	172.22.0.4	SSHv2	112	Server:
25	2.760602	172.22.0.4	172.22.0.5	TCP	68	44936 → 22 [ACK] Seq=1687 Ack=1795 Win=69376 Len=0 TSval=4233803577 TSecr=1398591111
26	2.760693	172.22.0.4	172.22.0.5	SSHv2	136	Client:
27	2.767904	172.22.0.5	172.22.0.4	SSHv2	120	Server:
28	2.808665	172.22.0.4	172.22.0.5	TCP	68	44936 → 22 [ACK] Seq=1755 Ack=1847 Win=69376 Len=0 TSval=4233803625 TSecr=1398591118
29	5.429452	172.22.0.4	172.22.0.5	SSHv2	216	Client:
30	5.470263	172.22.0.5	172.22.0.4	TCP	68	22 → 44936 [ACK] Seq=1847 Ack=1903 Win=71168 Len=0 TSval=1398593821 TSecr=4233806246
31	5.485738	172.22.0.5	172.22.0.4	SSHv2	96	Server:
32	5.485753	172.22.0.4	172.22.0.5	TCP	68	44936 → 22 [ACK] Seq=1903 Ack=1875 Win=69376 Len=0 TSval=4233806302 TSecr=1398593836
33	5.485882	172.22.0.4	172.22.0.5	SSHv2	180	Client:
34	5.485897	172.22.0.5	172.22.0.4	TCP	68	22 → 44936 [ACK] Seq=1875 Ack=2015 Win=71168 Len=0 TSval=1398593836 TSecr=4233806302
35	5.500876	172.22.0.5	172.22.0.4	SSHv2	696	Server:
36	5.541167	172.22.0.4	172.22.0.5	TCP	60	44936 → 22 [ACK] Seq=2015 Ack=2503 Win=69376 Len=0 TSval=4233806350 TSecr=1398593851

▶ Frame 10: 110 bytes on wire (880 bits), 110 bytes captured (880 bit	0000 00 04 00 01 00 06 02 42 ac 16 00 04 00 00 08 00B.....
▶ Linux cooked capture v1	0010 45 00 00 5e 7a 49 40 00 40 06 68 1b ac 16 00 04E..^zI@. @-h.....
▶ Internet Protocol Version 4, Src: 172.22.0.4, Dst: 172.22.0.5	0020 ac 16 00 05 af 88 00 16 f5 8d 22 f5 25 bc 23 ee".%#.....
▶ Transmission Control Protocol, Src Port: 44936, Dst Port: 22, Seq:	0030 80 18 01 f6 58 86 00 00 01 01 08 0a fc 5a ac 72X.....Z.r.....
▼ SSH Protocol	0040 53 5c c3 bf 53 53 48 2d 32 2e 30 2d 4f 70 65 6e S..SSH- 2.0-Open
Protocol: SSH-2.0-OpenSSH 8.2p1 Ubuntu-4ubuntu0.13	0050 53 53 48 5f 38 2e 32 70 31 20 55 62 75 6e 74 75 SSH.8.2p 1 Ubuntu
[Direction: client-to-server]	0060 2d 34 75 62 75 6e 74 75 30 2e 31 33 0d 0a -4ubuntu 0.13..

Figura 8: Tráfico obtenido al realizar conexión ssh.

En la captura se observa que se mantiene el funcionamiento explicado anteriormente, cambiando la versión OpenSSH del cliente, no así la del servidor ya que es el mismo. Si se inspecciona el paquete Key Exchange Init que contiene el HASSH del cliente se obtiene lo siguiente:

1.6 Tráfico generado por C4 (iface lo), detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

1 DESARROLLO (PARTE 1)

```

14 0.011049 172.22.0.4 172.22.0.5 SSHv2 1604 Client: Key Exchange Init
15 0.011063 172.22.0.5 172.22.0.4 TCP 68 22 → 44936 [ACK] Seq=43 Ar

```

▶ Frame 14: 1604 bytes on wire (12832 bits), 1604 bytes captured (12832 bits)
 ▶ Linux cooked capture v1
 ▶ Internet Protocol Version 4, Src: 172.22.0.4, Dst: 172.22.0.5
 ▶ Transmission Control Protocol, Src Port: 44936, Dst Port: 22, Seq: 43, Ack: 43, Len: 1536
 ▼ SSH Protocol
 ▼ SSH Version 2 (encryption:chacha20-poly1305@openssh.com compression:none)
 Packet Length: 1532
 Padding Length: 5
 ▼ Key Exchange (method:curve25519-sha256)
 Message Code: Key Exchange Init (20)
 ▼ Algorithms
 Cookie: e44ce792f1970264de5c5699cec85c3e
 kex_algorithms length: 270
 kex_algorithms string [truncated]: curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2
 server_host_key_algorithms length: 500
 server_host_key_algorithms string [truncated]: ecdsa-sha2-nistp256-cert-v01@openssh.com,ecd
 encryption_algorithms_client_to_server length: 188
 encryption_algorithms_client_to_server string: chacha20-poly1305@openssh.com,aes128-ctr,aes
 encryption_algorithms_server_to_client length: 188
 encryption_algorithms_server_to_client string: chacha20-poly1305@openssh.com,aes128-ctr,aes
 mac_algorithms_client_to_server length: 213
 mac_algorithms_client_to_server string [truncated]: umac-64-etm@openssh.com,umac-128-etm@op
 mac_algorithms_server_to_client length: 213
 mac_algorithms_server_to_client string [truncated]: umac-64-etm@openssh.com,umac-128-etm@op
 compression_algorithms_client_to_server length: 26
 compression_algorithms_client_to_server string: none,zlib@openssh.com,zlib
 compression_algorithms_server_to_client length: 26
 compression_algorithms_server_to_client string: none,zlib@openssh.com,zlib
 languages_client_to_server length: 0
 languages_client_to_server string:
 languages_server_to_client length: 0
 languages_server_to_client string:
 First KEX Packet Follows: 0
 Reserved: 00000000
 (hasshAlgorithms [truncated]: curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nist
 (hassh: c11b200866cf918393e62ea25d851d90)

Figura 9: HASSH C3.

Como se observa, el paquete tiene un total de 1604 Bytes, mientras que la respuesta del servidor tiene un total de 1180 Bytes. Este HASSH debería ser idéntico para todas las KEI que se realicen con la lista de algoritmos entregada por la versión de OpenSSH.

1.6. Tráfico generado por C4 (iface lo), detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Se captura el tráfico ssh de C4 al conectarse a C4-S1 y se obtiene lo siguiente:

1.6 Tráfico generado por C4 (iface lo), detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

1 DESARROLLO (PARTE 1)

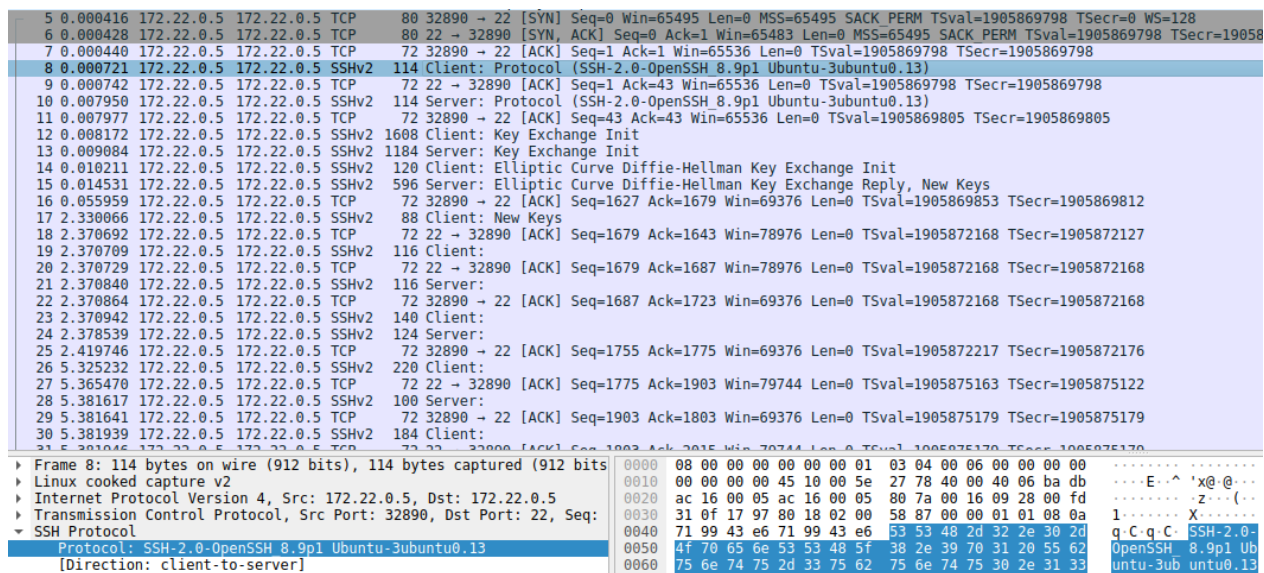


Figura 10: Tráfico obtenido al realizar conexión ssh.

En la captura se observa que se mantiene el funcionamiento explicado anteriormente, cambiando la versión OpenSSH del cliente, no así la del servidor ya que es el mismo. Si se inspecciona el paquete Key Exchange Init que contiene el HASSH del cliente se obtiene lo siguiente:

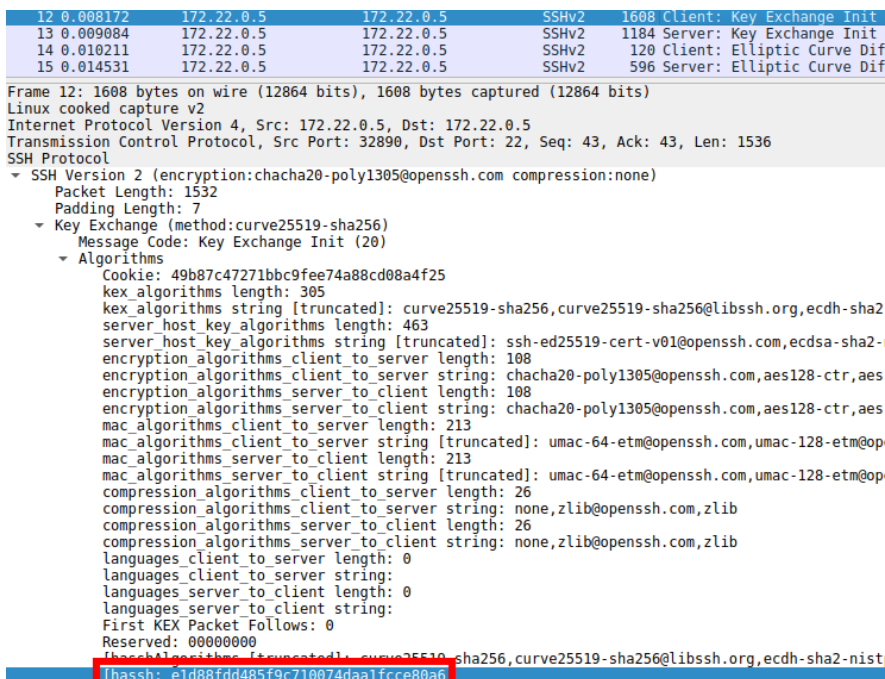


Figura 11: HASSH C4.

1.7 Tipo de información contenida en cada uno de los paquetes generados en texto plano

Como se observa, el paquete tiene un total de 1608 Bytes, mientras que la respuesta del servidor tiene un total de 1184 Bytes. Este HASSH debería ser idéntico para todas las KEI que se realicen con la lista de algoritmos entregada por la versión de OpenSSH.

1.7. Tipo de información contenida en cada uno de los paquetes generados en texto plano

1.7.1. C1

En c1 se observa en texto plano que primero el cliente indica que tiene la versión de OpenSSH 7.2p2 y el servidor responde que tiene la versión de OpenSSH 8.9p1, luego ambos intercambian la lista de algoritmos de cifrado que tienen disponibles y eligen el primero que tengan en común para realizar el proceso de Diffie-Hellman con sus llaves y parámetros públicos, que en este caso corresponde a una curva elíptica.

1.7.2. C2

En c1 se observa en texto plano que primero el cliente indica que tiene la versión de OpenSSH 7.6p1 y el servidor responde que tiene la versión de OpenSSH 8.9p1, luego ambos intercambian la lista de algoritmos de cifrado que tienen disponibles y eligen el primero que tengan en común para realizar el proceso de Diffie-Hellman con sus llaves y parámetros públicos, que en este caso corresponde a una curva elíptica.

1.7.3. C3

En c1 se observa en texto plano que primero el cliente indica que tiene la versión de OpenSSH 8.2p1 y el servidor responde que tiene la versión de OpenSSH 8.9p1, luego ambos intercambian la lista de algoritmos de cifrado que tienen disponibles y eligen el primero que tengan en común para realizar el proceso de Diffie-Hellman con sus llaves y parámetros públicos, que en este caso corresponde a una curva elíptica.

1.7.4. C4/S1

En C4-S1 ocurre algo particular ya que, el host del cliente es el mismo que el del servidor, por lo tanto se observa en texto plano que ambas partes tienen la versión de OpenSSH 8.9p1 y ambos tienen la misma lista de algoritmos de cifrado disponibles y eligen el primero que tengan en común para realizar el proceso de Diffie-Hellman con sus llaves y parámetros públicos, que en este caso corresponde a una curva elíptica.

1.8. Diferencia entre C1 y C2

Como se notó anteriormente, C1 y C2 cuentan con distintas versiones de Ubuntu y, en consecuencia, de OpenSSH. En este caso C1 tiene la versión 16.04 de Ubuntu y 7.2p2 de OpenSSH, mientras que C2 tiene la versión 18.04 de Ubuntu y 7.6p1 de OpenSSH. Esto

provoca en el paquete Protocol envíen distintas listas de algoritmos, provocando una variación en el tamaño del paquete. Esta diferencia también influye en el tamaño del paquete KEI (Key Exchange Init) donde se envían 1408 Bytes en C1 y 1428 en C2.

1.9. Diferencia entre C2 y C3

En este caso C2 y C3 también con distintas versiones de Ubuntu y, en consecuencia, de OpenSSH. En este caso C2 tiene la versión 18.04 de Ubuntu y 7.6p1 de OpenSSH, mientras que C3 tiene la versión 20.04 de Ubuntu y 8.2p1 de OpenSSH. Esto provoca en el paquete Protocol envíen distintas listas de algoritmos, provocando una variación en el tamaño del paquete. Esta diferencia también influye en el tamaño del paquete KEI (Key Exchange Init) donde se envían 1428 Bytes en C2 y 1604 en C3.

1.10. Diferencia entre C3 y C4

C3 y C4 también con distintas versiones de Ubuntu y, en consecuencia, de OpenSSH. En este caso C3 tiene la versión 20.04 de Ubuntu y 8.2p1 de OpenSSH, mientras que C4 tiene la versión 22.04 de Ubuntu y 8.9p1 de OpenSSH. Esto provoca en el paquete Protocol envíen distintas listas de algoritmos, provocando una variación en el tamaño del paquete. Esta diferencia también influye en el tamaño del paquete KEI (Key Exchange Init) donde se envían 1604 Bytes en C3 y 1608 en C4.

2. Desarrollo (Parte 2)

2.1. Identificación del cliente SSH con versión “?”

El cliente utilizado es un cliente personalizado, esto se descubre debido a que el cliente OpenSSH envía por defecto la versión que esta utilizando.

2.2. Replicación de tráfico al servidor (paso por paso)

Para replicar el tráfico se utiliza Python con la librería que permite implementar un cliente ssh y modificar la versión anunciada a la hora de realizar una conexión un servidor. Para esta actividad se utiliza el siguiente script de Python:

```
1 import paramiko
2 import socket
3 HOST_IP = "c4-s1"
4 PUERTO = 22
5 VERSION_A_FALSIFICAR = "SSH-2.0-OpenSSH_?"
6 USUARIO = "prueba"
7 PASSWORD = "prueba"
8
9 sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10 sock.connect((HOST_IP, PUERTO))
11 transport = paramiko.Transport(sock)
12 transport.local_version = VERSION_A_FALSIFICAR
13 transport.start_client()
14 transport.auth_password(username=USUARIO, password=PASSWORD)
```

Listing 5: Script de Python para falsificar versión de OpenSSH.

En este script, primero se definen los parámetros a utilizar, indicando la dirección IP (que en este caso se usa el nombre del contenedor gracias al DNS que implementa Docker), el puerto de la conexión, las credenciales y también el mensaje que se envía en texto plano al comenzar la conexión, que en este caso corresponde a "SSH-2.0-OpenSSH_?". Luego, se inicia una captura y se ejecuta el script, obteniendo el siguiente resultado en la conexión:

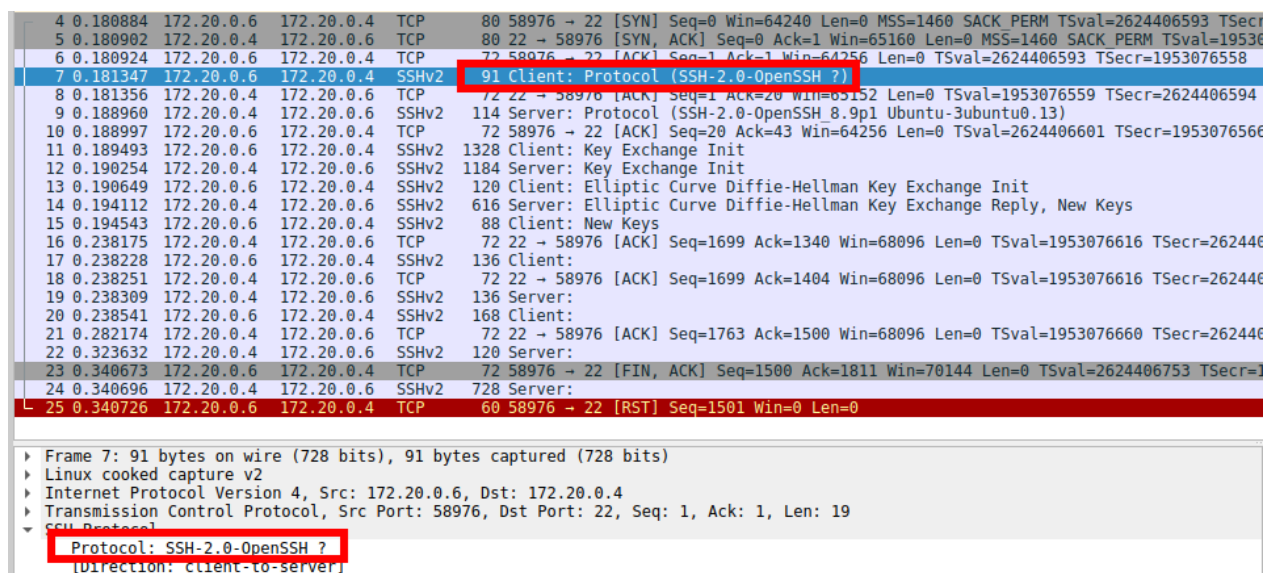


Figura 12: Tráfico obtenido al realizar conexión ssh.

Como se observa en la figura 12, la conexión se realiza con éxito y el paquete Protocol marca la versión solicitada en la actividad.

3. Desarrollo (Parte 3)

3.1. Replicación del KEI con tamaño menor a 300 bytes (paso por paso)

Como se vio en las actividades anteriores, el paquete Key Exchange Init por parte del servidor, es el encargado de entregar la lista de algoritmos de cifrado disponibles en el servidor. Por lo que para lograr reducir su tamaño desde 1184 Bytes hasta 300 Bytes, se realiza una reducción en la lista de algoritmos disponibles. Esta lista se encuentra en el archivo de configuración ssh disponible en la ruta `./etc/ssh/ssh_config` del contenedor del servidor y su configuración queda de la siguiente manera:

```
110
111 # Allow client to pass locale environment variables
112 AcceptEnv LANG LC_*
113
114 # override default of no subsystems
115 Subsystem sftp /usr/lib/openssh/sftp-server
116
117 # Example of overriding settings on a per-user basis
118 #Match User anoncvs
119 #   X11Forwarding no
120 #   AllowTcpForwarding no
121 #   PermitTTY no
122 #   ForceCommand cvs server
123 Ciphers aes128-ctr
124 MACs hmac-sha1
125 HostKeyAlgorithms ssh-ed25519
126 KexAlgorithms curve25519-sha256
127
```

Figura 13: Tráfico obtenido al realizar conexión ssh.

Al realizar una conexión capturando el tráfico se obtiene el siguiente resultado:

3.1 Replicación del KEI con tamaño menor a 300 bytes (parte DESARROLLO (PARTE 3))

13	132.700...	172.20.0.5	172.20.0.5	TCP	80	40364 → 22	[SYN] Seq=0 Win=65495 Len=0 MSS=6
14	132.700...	172.20.0.5	172.20.0.5	TCP	80	22 → 40364	[SYN, ACK] Seq=0 Ack=1 Win=65483
15	132.700...	172.20.0.5	172.20.0.5	TCP	72	40364 → 22	[ACK] Seq=1 Ack=1 Win=65536 Len=0
16	132.700...	172.20.0.5	172.20.0.5	SSHv2	114	Client:	Protocol (SSH-2.0-OpenSSH 8.9p1 Ubur
17	132.700...	172.20.0.5	172.20.0.5	TCP	72	22 → 40364	[ACK] Seq=1 Ack=43 Win=65536 Len=0
18	132.709...	172.20.0.5	172.20.0.5	SSHv2	114	Server:	Protocol (SSH-2.0-OpenSSH 8.9p1 Ubur
19	132.709...	172.20.0.5	172.20.0.5	TCP	72	40364 → 22	[ACK] Seq=43 Ack=43 Win=65536 Len=0
20	132.709...	172.20.0.5	172.20.0.5	SSHv2	1608	Client:	Key Exchange Init
21	132.711...	172.20.0.5	172.20.0.5	SSHv2	280	Server:	Key Exchange Init
22	132.713...	172.20.0.5	172.20.0.5	SSHv2	120	Client:	Elliptic Curve Diffie-Hellman Key Ex
23	132.716...	172.20.0.5	172.20.0.5	SSHv2	604	Server:	Elliptic Curve Diffie-Hellman Key Ex
24	132.758...	172.20.0.5	172.20.0.5	TCP	72	40364 → 22	[ACK] Seq=1627 Ack=783 Win=65536
25	17.1663...	172.20.0.5	172.20.0.5	SSHv2	88	Client:	New Keys
26	17.2089...	172.20.0.5	172.20.0.5	TCP	72	22 → 40364	[ACK] Seq=783 Ack=1643 Win=81024
27	17.2089...	172.20.0.5	172.20.0.5	SSHv2	124	Client:	

Frame 21: 280 bytes on wire (2240 bits) 280 bytes captured (2240 bits)

Linux cooked capture v2

Internet Protocol Version 4, Src: 172.20.0.5, Dst: 172.20.0.5

Transmission Control Protocol, Src Port: 22, Dst Port: 40364, Seq: 43, Ack: 1579, Len: 208

SSH Protocol

- SSH Version 2 (encryption:aes128-ctr compression:none)
 - Packet Length: 204
 - Padding Length: 4
 - Key Exchange (method:curve25519-sha256)
 - Message Code: Key Exchange Init (20)
 - Algorithms
 - Cookie: 0989dc9960c9aefea3b14ab4ac459a
 - kex_algorithms length: 46
 - kex_algorithms string: curve25519-sha256,kex-strict-s-v00@openssh.com
 - server host key algorithms length: 11
 - server host key algorithms string: ssh-ed25519
 - encryption_algorithms_client_to_server length: 10
 - encryption_algorithms_client_to_server string: aes128-ctr
 - encryption_algorithms_server_to_client length: 10
 - encryption_algorithms_server_to_client string: aes128-ctr
 - mac_algorithms_client_to_server length: 9
 - mac_algorithms_client_to_server string: hmac-sha1
 - mac_algorithms_server_to_client length: 9
 - mac_algorithms_server_to_client string: hmac-sha1
 - compression_algorithms_client_to_server length: 21
 - compression_algorithms_client_to_server string: none,zlib@openssh.com
 - compression_algorithms_server_to_client length: 21
 - compression_algorithms_server_to_client string: none,zlib@openssh.com
 - languages_client_to_server length: 0
 - languages_client_to_server string:
 - languages_server_to_client length: 0
 - languages_server_to_client string:
 - First KEX Packet Follows: 0
 - Reserved: 00000000
 - [hasshServerAlgorithms: curve25519-sha256,kex-strict-s-v00@openssh.com;aes128-ctr
 - [hasshServer: e6c4b4dcf9fd5d63aa8cf8f0d3079bed]

Figura 14: Tráfico obtenido al realizar conexión ssh.

4. Desarrollo (Parte 4)

4.1. Explicación OpenSSH en general

OpenSSH es un software de código abierto que implementa el protocolo SSH (Secure Shell). El protocolo SSH funciona con cliente-servidor. Como se observó en este laboratorio, se tenía un servidor S1 y múltiples clientes C1, C2, C3 Y C4. El proceso de funcionamiento, visto en las capturas de Wireshark, comienza con un handshake donde cliente y servidor intercambian sus versiones de protocolo. Después, ocurre el Key Exchange Init (KEI). Este paquete, contiene el HASSH y ambas partes negocian los algoritmos criptográficos que usarán para intercambiar claves, para cifrar simétricamente, etc.

Una vez negociados, establecen una clave de sesión simétrica y cifran todo el tráfico posterior, creando un túnel a través de una red que de otra manera sería insegura.

4.2. Capas de Seguridad en OpenSSH

El protocolo OpenSSH garantiza los principios de seguridad mediante una arquitectura de varias capas, muchas de las cuales son visibles en el tráfico interceptado (específicamente en los paquetes KEI):

- **Confidencialidad:** Se logra mediante cifrado simétrico (AES-256, ChaCha20). Una vez que el cliente y el servidor completan el intercambio de claves Diffie-Hellman, generan un secreto compartido. Esta clave es usada para cifrar todo el contenido de la comunicación posterior. Esto garantiza que un atacante que intercepte el tráfico no pueda leer los comandos ejecutados.
- **Integridad:** Se garantiza mediante el uso de MAC, como HMAC-SHA2. Cada paquete enviado incluye un MAC, que es un hash criptográfico del contenido del paquete, firmado con el secreto compartido. El receptor recalcula el MAC y lo compara con el recibido. Si no coinciden, el paquete se descarta. Esto protege contra la modificación de datos en tránsito (MITM). Los algoritmos MAC también se negocian en el paquete KEI.
- **Autenticidad:** Este principio se aplica en el cliente y en el servidor:
 1. **Autenticación del Servidor:** El servidor debe probarle al cliente que es quien dice ser. Lo hace utilizando su clave de host (host key). Durante el handshake, el servidor firma una parte del intercambio con su clave privada. El cliente verifica esta firma con la clave pública del servidor.
 2. **Autenticación del Usuario:** El cliente debe probar su identidad al servidor. En este caso se utilizó el método de autenticación por contraseña. El cliente envía la contraseña "prueba."^a través del túnel ya cifrado.

4.3. Identificación de que protocolos no se cumplen

Basado en el análisis anterior y el tráfico interceptado, los principios que no se cumplen o se cumplen débilmente en la configuración de este laboratorio son:

1. **No Repudio:** Al usar la autenticación por contraseña (en vez de clave pública y privada) y una contraseña fácil de robar como "prueba", cualquier persona con esa contraseña puede actuar como el usuario. El servidor no tiene forma criptográfica de probar que fue el usuario legítimo y no un impostor quien inició sesión y ejecutó comandos. El protocolo, en esta modalidad, no puede evitar que el usuario (o un atacante) repudie.^o niegue haber realizado una acción.
2. **Disponibilidad (Availability):** Este principio no es una responsabilidad directa del protocolo OpenSSH, sino de la implementación y la infraestructura que lo soporta. El protocolo SSH en sí no ofrece protección inherente contra ataques de Denegación de Servicio que sature el servicio sshd. Si bien el servicio estaba disponible en Docker, su disponibilidad no está garantizada por el protocolo mismo y sigue siendo vulnerable a ataques de red o a fallos en la implementación del servicio en el contenedor.

Conclusiones y comentarios

Finalmente, se concluye que se logra realizar con éxito cada una de las actividades planteadas, configurando correctamente los 4 clientes y el servidor, realizando las conexiones en cada cliente, implementando un cliente personalizado con Python para cambiar la versión anunciada en texto plano y configurando el archivo de configuración del servidor para reducir el tamaño del paquete KEI enviado por el mismo. Además se estudió a fondo el protocolo ssh en la implementación de OpenSSH, analizando que principios de seguridad cumple y cuales no.